

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 1
MODUL 18
“MESIN ABSTRAK”



DISUSUN OLEH:
RAFI RAMADHAN
109082500140
S1 IF-13-02
DOSEN:
Wahyu Andi Saputra, S.Pd., M.Eng.

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2024/2025

DASAR TEORI

Dalam pembangunan perangkat lunak, sebuah program dapat dipandang sebagai model komputasi yang terdiri atas data yang akan diolah serta sekumpulan operasi dasar untuk mengolah data tersebut. Salah satu pendekatan dalam merancang model komputasi adalah dengan menggunakan konsep mesin abstrak (abstract machine). Mesin abstrak merupakan model komputasi yang dibangun di atas mesin komputasi lain yang sudah tersedia sebelumnya, di mana tipe data dan operasi-operasi dasarnya diimplementasikan dengan memanfaatkan tipe data dan operasi yang sudah ada pada mesin di bawahnya (lower-level machine). Dengan demikian, mesin abstrak menyembunyikan detail implementasi tingkat rendah dan menyediakan antarmuka yang lebih sesuai dengan kebutuhan persoalan yang ingin diselesaikan.

Pendekatan ini memungkinkan sebuah sistem yang kompleks dibangun secara bertahap melalui beberapa lapisan abstraksi. Setiap lapisan hanya bergantung pada operasi yang disediakan oleh lapisan di bawahnya, tanpa perlu mengetahui bagaimana operasi tersebut diimplementasikan secara internal. Teknik penyusunan lapisan abstraksi seperti ini menjadi salah satu cara untuk mengelola kompleksitas dalam pengembangan perangkat lunak, sekaligus memudahkan perancangan algoritma karena programmer dapat berfokus pada logika penyelesaian masalah tanpa terganggu oleh detail teknis tingkat rendah.

Salah satu contoh penerapan konsep mesin abstrak adalah mesin abstrak untuk permainan kartu domino. Mesin ini bekerja pada tipe data kartu domino, yaitu kartu dengan dua sisi yang masing-masing memiliki titik (pip) bernilai 0 hingga 6, sehingga terdapat 28 kartu unik tanpa duplikasi. Mesin abstrak domino menyediakan operasi dasar seperti kocok kartu untuk mengacak urutan kartu, ambil kartu untuk mengambil satu kartu dari tumpukan, gambar kartu untuk mengetahui sisi (suit) tertentu dari sebuah kartu, dan nilai kartu untuk menghitung nilai total kartu. Operasi-operasi dasar ini menjadi fondasi untuk membangun operasi yang lebih kompleks, misalnya memeriksa apakah suatu kartu merupakan kartu balak, mencari pasangan kartu dengan total nilai tertentu, hingga mengimplementasikan aturan permainan seperti Gapleh, Kiu-kiu, Luzon, dan Texas 42.

Selain pada domain permainan kartu, konsep mesin abstrak juga dapat diterapkan pada pengolahan untaian karakter melalui mesin abstrak karakter. Mesin ini bekerja pada suatu rangkaian karakter yang diakhiri dengan tanda titik sebagai penanda akhir, dan menyediakan empat operasi dasar, yaitu `start()` untuk menempatkan pembaca pada posisi awal rangkaian, `maju()` untuk memindahkan pembaca ke karakter selanjutnya, `eop()` untuk memeriksa apakah pembaca telah mencapai akhir rangkaian, dan `cc()` untuk membaca karakter pada posisi pembaca saat ini. Dengan keempat operasi dasar tersebut, berbagai algoritma pengolahan string dapat disusun, seperti menghitung jumlah karakter yang dibaca, menghitung jumlah kemunculan huruf tertentu, menghitung frekuensi kemunculan suatu huruf terhadap total karakter, maupun mendeteksi pola pasangan huruf tertentu dalam rangkaian karakter yang dibaca.

SOAL LATIHAN

Statement perulangan

1. A. Tipe data kartu domino

Source Code:

```
type Domino struct {
    Sisi1 int
    Sisi2 int
    Nilai int
    Balak bool
}

type Dominoes struct {
    Kartu []Domino
    Jumlah int
}

func BuatDomino(s1, s2 int) Domino {
    return Domino{
        Sisi1: s1,
        Sisi2: s2,
        Nilai: s1 + s2,
        Balak: s1 == s2,
    }
}

func BuatSetDomino() Dominoes {
    var D Dominoes
    for i := 0; i <= 6; i++ {
        for j := i; j <= 6; j++ {
            D.Kartu = append(D.Kartu, BuatDomino(i, j))
        }
    }
    D.Jumlah = len(D.Kartu)
    return D
}
```

Deskripsi Program:

Bagian ini mendefinisikan struktur data fundamental menggunakan struct untuk merepresentasikan sebuah kartu domino individual dan sekumpulan tumpukan kartu. Struktur Domino menyimpan informasi titik pada kedua sisi, total nilai, serta status balak, sementara fungsi pembuat set domino menggunakan perulangan bersarang untuk menggenerasi 28 kombinasi kartu secara otomatis dan menyimpannya ke dalam sebuah *slice*.

B. kocokKartu

Source Code:

```
func kocokKartu(D *Dominoes) {
    for i := range D.Kartu {
        j := rand.Intn(len(D.Kartu))
        D.Kartu[i], D.Kartu[j] = D.Kartu[j], D.Kartu[i]
    }
}
```

```
}  
}
```

Deskripsi Program:

Prosedur pengocokan kartu ini memanfaatkan fungsi *random* bawaan untuk mengacak urutan elemen di dalam *slice* kartu domino. Algoritma berjalan dengan melakukan iterasi pada seluruh kartu yang ada di tumpukan, lalu menukar posisi (*swap*) setiap kartu dengan kartu lain yang berada di indeks acak, sehingga menghasilkan susunan tumpukan yang tidak dapat diprediksi.

C. ambilKartu

Source Code:

```
func ambilKartu(D *Dominoes) Domino {  
    if D.Jumlah == 0 {  
        return Domino{}  
    }  
    D.Jumlah--  
    kartu := D.Kartu[D.Jumlah]  
    D.Kartu = D.Kartu[:D.Jumlah]  
    return kartu  
}
```

Deskripsi Program:

Fungsi pengambilan kartu dirancang untuk menyimulasikan proses menarik satu kartu teratas dari tumpukan dengan mengelola nilai jumlah kartu secara dinamis. Jika tumpukan kosong, fungsi mengembalikan kartu kosong, namun jika masih ada, sistem akan mengurangi nilai jumlah kartu, mengambil elemen pada indeks terakhir *slice*, dan memotong rentang *slice* tersebut agar kartu benar-benar hilang dari tumpukan.

D. gambarKartu

Source Code:

```
func gambarKartu(d Domino, suit int) int {  
    if suit == 1 {  
        return d.Sisi1  
    }  
    return d.Sisi2  
}
```

Deskripsi Program:

Fungsi ini bertugas sebagai pengakses (selektor) nilai spesifik dari sebuah kartu domino berdasarkan sisi atau "suit" yang ingin diperiksa oleh sistem. Dengan menerima parameter angka penentu (1 atau 2), algoritma menggunakan seleksi kondisi sederhana untuk mengembalikan nilai pada Sisi1 jika parameter bernilai 1, dan mengembalikan Sisi2 untuk kondisi lainnya.

E. nilaiKartu

Source Code:

```
func nilaiKartu(d Domino) int {  
    return d.Nilai  
}
```

Deskripsi Program:

Operasi dasar ini adalah sebuah fungsi *getter* ringkas yang berfungsi untuk mengambil dan mengembalikan total titik dari sebuah kartu domino tunggal. Alih-alih menghitung ulang jumlah kedua sisi setiap kali dipanggil, baris kode ini langsung mengakses atribut Nilai dari struktur data yang sudah dikalkulasi di awal saat kartu pertama kali dicetak.

2. A. Fungsi galiKartu

Source Code:

```
func galiKartu(D *Dominoes, target Domino) Domino {  
    for D.Jumlah > 0 {  
        kartu := ambilKartu(D)  
        if kartu.Sisi1 == target.Sisi1 ||  
           kartu.Sisi2 == target.Sisi1 ||  
           kartu.Sisi1 == target.Sisi2 ||  
           kartu.Sisi2 == target.Sisi2 {  
            return kartu  
        }  
    }  
    return Domino{}  
}
```

Deskripsi Program:

Algoritma pencarian ini mensimulasikan proses menarik kartu dari tumpukan secara terus-menerus hingga ditemukan kartu yang memiliki angka (titik) yang cocok dengan salah satu sisi dari kartu target. Menggunakan perulangan yang berjalan selama tumpukan belum kosong, fungsi mengevaluasi setiap kartu dengan operator logika OR (||); jika salah satu sisi cocok, perulangan berhenti dan kartu dikembalikan.

B. Fungsi sepasangKartu

Source Code:

```
func sepasangKartu(d1, d2 Domino) bool {  
    total := nilaiKartu(d1) + nilaiKartu(d2)  
    return total == 12  
}
```

Deskripsi Program:

Fungsi ini merupakan predikat logika yang mengembalikan nilai *boolean* untuk mengevaluasi apakah gabungan dari dua buah kartu domino memenuhi syarat angka mutlak tertentu. Program memanggil operasi dasar nilaiKartu untuk masing-masing parameter kartu, menjumlahkannya, lalu membandingkan total tersebut dengan angka 12, yang mana akan menghasilkan kondisi true jika persis bernilai 12.

3. Simulasi Permainan Gapleh

Source Code:

```

func mainkanGapleh(D *Dominoes) {
    if D.Jumlah == 0 {
        return
    }

    fmt.Println("Kartu Awal:")
    fmt.Printf("(%d,%d)\n", awal.Sisi1, awal.Sisi2)

    ujung := awal.Sisi2

    for D.Jumlah > 0 {
        kartu := ambilKartu(D)

        if kartu.Sisi1 == ujung {
            fmt.Printf("Pasang (%d,%d)\n", kartu.Sisi1,
kartu.Sisi2)
            ujung = kartu.Sisi2
        } else if kartu.Sisi2 == ujung {
            fmt.Printf("Pasang (%d,%d)\n", kartu.Sisi1,
kartu.Sisi2)
            ujung = kartu.Sisi1
        } else {
            fmt.Printf("Buang (%d,%d)\n", kartu.Sisi1,
kartu.Sisi2)
        }
    }
}

```

Deskripsi Program:

Prosedur ini merangkai seluruh operasi dasar domino menjadi sebuah alur algoritma sekuensial yang utuh untuk bermain di atas meja. Dimulai dengan meletakkan satu kartu awal dan menetapkan sisi keduanya sebagai "ujung", program menggunakan perulangan untuk menarik sisa kartu dan mencocokkannya; jika sisi kartu cocok dengan ujung meja, kartu terpasang dan nilai ujung diperbarui secara dinamis.

4. A. Operasi Dasar Mesin Karakter

Source Code:

```

var pita string
var idx int
var currentChar byte

func start(input string) {
    pita = input
    idx = 0
    if len(pita) > 0 {
        currentChar = pita[idx]
    }
}

func maju() {
    idx++
    if idx < len(pita) {
        currentChar = pita[idx]
    }
}

```

```

func eop() bool {
    return currentChar == '.'
}

func cc() byte {
    return currentChar
}

```

Deskripsi Program:

Bagian ini membangun pondasi mesin pembaca teks menggunakan variabel global untuk menyimpan untaian pita, indeks posisi, dan karakter yang sedang dibaca. Empat fungsi utamanya bekerja menyerupai mesin pemutar: start menginisialisasi pembacaan di indeks nol, maju menggeser indeks satu langkah ke depan, cc mengambil karakter di posisi saat ini, dan eop (*End of Process*) bertindak sebagai pembatas yang merespons true saat menabrak karakter titik.

B. Implementasi Algoritma Mesin Karakter

Source Code:

```

func bacaSemuaKarakter(input string) {
    start(input)
    fmt.Print("Karakter terbaca : ")
    for !eop() {
        fmt.Printf("%c", cc())
        maju()
    }
    fmt.Println()
}

func hitungKarakter(input string) int {
    start(input)
    jumlah := 0
    for !eop() {
        jumlah++
        maju()
    }
    return jumlah
}

func hitungA(input string) int {
    start(input)
    jumlahA := 0
    for !eop() {
        if cc() == 'A' {
            jumlahA++
        }
        maju()
    }
    return jumlahA
}

func frekuensiA(input string) float64 {
    total := hitungKarakter(input)
    jumlahA := hitungA(input)
    if total == 0 {
        return 0
    }
    return float64(jumlahA) / float64(total)
}

```

```

func hitungLE(input string) int {
    start(input)
    jumlah := 0
    var prev byte = ' '

    for !eof() {
        if prev == 'L' && cc() == 'E' {
            jumlah++
        }
        prev = cc()
        maju()
    }
    return jumlah
}

```

Deskripsi Program:

Bagian ini merupakan implementasi terapan dari operasi dasar mesin karakter untuk melakukan berbagai pemrosesan teks, mulai dari membaca hingga menganalisis pola kemunculan huruf.

DAFTAR PUSTAKA

School of Computing, Fakultas Informatika, Telkom University. *Modul Praktikum Algoritma dan Pemrograman 2 – Modul 18: Mesin Abstrak*.

Munir, R. *Spesifikasi dan Implementasi Tipe Data – Mesin Abstrak*. Bahan Kuliah Strategi Algoritma (IF2211), Program Studi Teknik Informatika, Institut Teknologi Bandung.

Anggreani, S. (2024). *Pengantar dan Pengenalan Konsep Dasar Algoritma pada Pembelajaran Pemrograman Komputer*. Jurnal Pendekar.

Mirolu, C. (2021). *Abstraction in Computer Science Education: An Overview*. Informatics in Education, 21(1).

Santoso, J. T. (2023). *Algoritma dan Teori Komputasi Terukur dalam Pembelajaran Mesin*. Yayasan Prima Agus Teknik.

Sulasromo, A. H. (2022). *Buku Ajar Algoritma dan Pemrograman I*. Penerbit P4I.

Fauzi, D., Hanim, H., Elwirehardja, G. N., Isnani, M., & Ihksan, M. (2024). *Fondasi Ilmu Komputer dari Algoritma hingga Kecerdasan Buatan*.

Putra, F. M. (2025). *Sebuah Tinjauan melalui Contoh Mesin Penjual Otomatis*. Blantika Multidisciplinary Journal.

Mengidentifikasi Dasar Algoritma Pemrograman. (2024). Jurnal UISIK.